Mini-Project 4: OOP for Data Analysis

Goals

The goal of the next two mini-projects will be to develop a sensor dashboard that can be used to streamline analysis of data collected using smartphone sensors. In this mini-project we will create a programmatic interface to the dashboard. In the following mini-project, we will develop a graphical user interface (GUI) that will allow users without a programming background to interact with our dashboard.

You will create your own class that *wraps* the existing Matlab mobiledev class (what we've been using to collect and analyze sensor data) and adds your own custom methods for analysis and visualization. Concepts from lecture that will be applied in this mini-project include:

- Object oriented programming: classes, methods, properties
- Conditional branches
- Numerical integration

Steps

In this mini-project you will create a custom class "SensorDataAnalyzer" that wraps the mobiledev class and adds the following methods:

- **logSensorData(timeSeconds, delaySeconds)** - runs a logging session lasting for "timeSeconds" after a delay of "delaySeconds."
- **setTargetSignal(signalName)** - selects the signal to be analyzed/visualized.
- **plotTargetSignal()** - plots the target signal.
- **getTargetSignal()** - returns the target signal and time stamp.
- **integrate()** - uses the trapezoidal method to numerically integrate the target signal over its domain.

A skeleton class file is provided on Canvas. The following steps will guide you through implementing and validating each of these methods.

Step 1: Create Class and add basic methods

Download the provided "SensorDataAnalyzer.m" skeleton file from Canvas. You will fill in this file throughout the remaining steps. In this step, we will add functionality to set the target signal to be analyzed. Perform the following steps:

1. Fill in the constructor method so that it creates a mobiledev connection and assigns the result to the mobiledevConnection property of the class. Modify the mobiledevConnection property so that it is a *private* property. Private properties of a class can be accessed/modified only by the methods of that class. It's a good practice to make properties storing internal state private so they cannot be accidentally modified by the user in a way that corrupts the functioning of your code.

2. Fill in the method **logSensorData(timeSeconds, delaySeconds)** so that, when called, your class logs sensor data for "timeSeconds" after a delay of "delaySeconds." As you've probably noticed, adding a delay is helpful so that the user has time to get set up for a data collection run.

3. Now, add a new method (no skeleton is provided) **setTargetSignal(signalName)** which takes a string "signalName" and, based on the value of this argument, extracts the appropriate signal from the logged sensor data and stores it in a *public* property called "TargetSignal". Store the timestamps in a public property called "TargetSignalTs"

   Important: your property *must* be called "TargetSignal" and it *must* be public. Possible values for signalName are: "X-Accel, Y-Accel, Z-Accel, R-Accel", where "R-Accel" indicates resultant acceleration. For example, if the value of signalName is "R-Accel," then you should calculate the resultant acceleration and store it in the TargetSignal property.
   a. Using the error function (https://www.mathworks.com/help/matlab/ref/error.html), throw an error, along with an informative message, if the passed signal name is not one of the options listed above or if the user has not run a logging session yet.
   b. Hints: You will need to use conditional branches to extract and set the appropriate signal. To check if a logging session has been run, you can check if "accellog" returns an empty array using the "isempty" function.

4. Finally, add a method called **plotTargetSignal(obj)** that plots TargetSignal on the Y axis and Timestamps on the X-axis. Your method should throw an error if TargetSignal has not been set yet (you can again do this using the isempty function).

Step 2: Add Numerical Integration

In this step, you will add functionality to your class to perform numerical integration of the target signal, and use this to perform *dead reckoning*: estimation of an object's position from measurements of its acceleration.

Background: Dead Reckoning

Dead reckoning is a simple procedure that can be used to estimate an object's position from measurements of its acceleration or velocity. Dead reckoning is appealing because it's very cheap and simple, but it's not very accurate. We'll work through a simple dead reckoning example that assumes objects are traveling in a straight line along a coordinate axis.

Dead reckoning is motivated by simple intuition from calculus: recall that an object's average velocity is its change in position over an interval of time, and its average acceleration is its change in velocity over an interval of time. If we write position of an object (in 1D space) as a function $x(t)$ of time and take a limit as the interval of time shrinks to zero, we can describe this more formally using calculus:

- Velocity: $v(t) = x'(t)$
- Acceleration: $a(t) = v'(t) = x''(t)$

Now, suppose we know $a(t)$ and want to calculate $x(t)$. The fundamental theorem of calculus tells us that:

$v(t) = v(0) + \int_0^t a(s)\, ds.$

$$x(t) = x(0) + \int_0^t v(s)\, ds$$

Therefore, given known initial conditions $x(0), v(0)$ (which we assume in this lab to be zero), we can calculate position by integrating acceleration twice: once to find velocity, and again to find position. Of course, we don't know $a(t)$ exactly, but we do get to see *samples* from this function collected by our accelerometer. We can numerically integrate these samples to *estimate* $x(t)$.

Step 2.1: Implementation

Add a method to your class with the signature **integrate(obj)** that performs numerical integration of the target signal using the cumtrapz function. Your method should overwrite the target signal with the result and, as before, must throw an error if no target signal has been set yet.

Hint: be careful to account for your sampling rate when performing numerical integration using cumtrapz. Take a close look at the section titled "Cumulatively Integrate Vector Data with Nonuniform Spacing" in the documentation for cumtrapz. The spacing of your integration will be determined by the timestamps of the collected measurements.

Note: In general, one should be careful to specify the initial conditions for numerical integration as shown in the formulae above. Here, we're assuming the initial conditions are zero, but, in general, it's good practice to set the initial conditions explicitly.

Step 2.2: Validation

We'll now use numerical integration to estimate your walking trajectory ("dead reckoning") from accelerometer data.

Data Collection Procedure

1. Hold your phone securely with the positive X-axis pointing forward.
2. Using your logSensorData method to collect data, in a clear, straight path (e.g., a hallway), start from rest, walk forward for a few paces, then walk back to exactly where you started (or at least as close as humanly possible).
3. Set the X-axis acceleration as your target signal.

Analysis Steps

1. Integrate the smoothed acceleration once to get velocity vs. time.
    - Think about what you expect the starting and ending value for velocity to be.
    - Now make a plot over estimated velocity over time and include it in your lab report. Does the plot match your expectations? If not, speculate as to why not.
2. Integrate the velocity to get displacement vs. time.
    - Produce a plot showing displacement over time and include it in your lab report.
    - Does the estimated distance walked match your actual distance? If not, speculate as to why not?

Step 3: Optional! Dead Reckoning Challenge

Note: This section is optional and is not required for full credit on the lab.

Now that you've implemented a basic dead-reckoning pipeline, it's time to compete! In this section, you'll work to improve the accuracy of your dead-reckoning pipeline and compete with your classmates to develop the most accurate approach.

Step 3.1: Brainstorming

Start by brainstorming at least three ways you can improve the accuracy of your pipeline. Here are some questions you might want to consider: are there any un-accounted for external forces acting on your phone's accelerometer that may be confounding results? Are the assumptions of the procedure described above (axis aligned motion starting from rest) perfectly satisfied in practice?

Step 3.2: Competition

Now, improve your pipeline to address the issues you identified above. Submit a supplement to your lab report (called "dead reckoning challenge") which describes the improvements you

made. Include a plot showing the results of your pipeline on the “out-and-back” task described above. Clearly state the final error in estimation of displacement from your initial position.

Rules:

1. You may modify any part of your processing pipeline and use any additional sensors (other than GPS) as long as you still use the raw acceleration data collected during the walk as your primary input.
2. You may **not** hard-code the known distance — your code should work on any short walk in a straight corridor.
3. Submit your final estimated displacement (smoothed and integrated) along with your code.
4. The five most accurate implementations will compete head-to-head in a live demo in class!
5. The winner will be the student with the smallest **absolute error** compared to the measured ground-truth displacement.

Prize: the winner will receive eternal glory in the annals of ENG6 lore and an Arduino Uno development kit.

What to Submit:

- Upload your SensorDataAnalyzer class file to the autograder on Canvas.
- Submit your lab report to the Mini-Project-4 assignment on Canvas. Your lab report should contain the following:
    - Plots of acceleration, estimated velocity, and estimated position for dead reckoning performed using numerical integration along with a short reflection on whether the plots matched your expectations.

Rubric

- Code is functional:
    - Target signal can be set/modified
    - Error messages are thrown as described in the lab manual
    - Numerical integration is performed correctly
    - Plotting is performed correctly on the target signal
- Numerical integration results are correct:
    - Velocity is estimated correctly and is consistent with accelerometer measurements.
    - Displacement (position) is estimated correctly and final estimated net displacement is reasonably close to zero.